2023 9th International Conference on Computer and Communications

LightRPC: An Efficient and Simple RDMA-based RPC Framework

Xingyu Wang
School of computer science and Engineering
Sun Yat-sen University
Guangzhou 510006, China
wangxy323@mail2.sysu.edu.cn

Dan Huang
School of computer science and Engineering
Sun Yat-sen University
Guangzhou 510006, China
huangd79@mail.sysu.edu.cn

2023 9th International Conference on Computer and Communications (ICCC) | 979-8-3503-1725-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCC59590.2023.10507594

***Abstract*—RDMA has been widely used in modern data centers because it can provide low-latency and high-bandwidth network communication for distributed applications. However, how to build an efficient RDMA-based RPC framework is still challenging. We observe that existing work may have the following problems: First, the lack of a serialization module makes the RPC framework difficult to use, and cannot further be optimized by overlapping serialization and network communication. Second, CPU resources are ignored to scale out instead of focusing on QP and memory resources, which may result in the RPC framework not being scalable. In this paper, we propose LightRPC, which provides users with easy-to-use and portable interfaces via a new resource abstraction and achieves efficient performance through co-design of serialization and communication, usage of pre-registered memory, and adaptive polling mode. Compared to bRPC's IPoIB and RDMA modes, LightRPC improves throughput by about 2.5x and 1.5x, respectively, and has lower median and tail latency.**

***Index Terms*—RDMA, RPC, Scalability**

I. Introduction

As an important high-performance network technology, RDMA has the features of kernel bypass and CPU offloading, thereby enabling low-latency and high-throughput network communication. Due to its excellent performance, RDMA has been widely used in distributed applications such as storage systems [1], transaction processing systems [2], and deep learning systems [3]. At the same time, except for scenarios where collective communication is required, most distributed applications use an RPC runtime to achieve more flexible point-to-point communication. The RPC programming model abstracts data interaction into function calls, hiding the underlying communication details so that users only need to care about the data to be sent and received, so its semantics are quite simple.

In fact, collective communication libraries such as OpenMPI and NCCL already have provided mature RDMA functions. However, there is still lacking of sophisticate RDMA-based RPC framework in the industry. In the past time, there has been a lot of research work on implementing RPC based on RDMA. They achieve high performance by selecting appropriate transmission modes and RDMA operations, and some of them make efforts to solve the non-scalability problem of RDMA. We summarize some reasons why these frameworks are not actually used: (1) Most frameworks do not have a serialization module, such as eRPC [4] and Flock [5]. This will bring about two problems: First, users may need to implement serialization by themselves, leading to increasing the burden of program refactoring. Second, collaboratively optimizing serialization and communication can not be implemented. (2) Some work ignores the management of CPU resources when reducing QP or memory usage. For example, RFP [6] requires the client to continuously use READ operations to obtain RPC responses, resulting in a waste of client CPU resources. RFP will not be available when there are multiple connections between client and server. As another example, ScaleRPC [7] assigns a busy polling thread to each client on the server side. When there are many clients, the CPU resources of the server will be exhausted.

Therefore, we design an efficient and easy-to-use RDMA-based RPC framework called LightRPC. We propose the following designs for resolving the existing problems: (1) Design a serialization module and setup pre-registered memory for message transmission. By serializing data objects directly into pre-registered memory, we achieve zero copy of user-mode memory. By implementing a lock-free queue to manage memory blocks, we achieve efficient application and return in high-concurrency scenarios. (2) We design an adaptive polling mode for the server. When no RPC requests arrive, the polling thread is in a blocked state, thus saving CPU resources. Once an RPC request arrives, the polling thread switches to the busy polling state to quickly sense the request and process it. (3) We provide users with simple interfaces by a new resource abstraction. Users can build RPC endpoints and make RPC calls with only a few lines of code. In conclusion, this paper mainly makes the following contributions:

- We analyze the technical problems that prohibit the existing RPC frameworks from being used in production-level applications.
- We design an RDMA-based RPC framework called LightRPC, which has a serialization module and achieves efficient performance through various optimizations.
- Our evaluation shows that compared with bRPC (released by Baidu), LightRPC can achieve higher throughput and lower latency in multi-connection scenarios.

The rest of this paper is organized as follows. We introduce

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on September 15,2026 at 06:53:28 UTC from IEEE Xplore. Restrictions apply.

the research background and design challenges in Section II. Section III presents LightRPC in detail, as well as the solutions to address the challenges. We evaluate LightRPC in Section IV and discuss related work in Section V.

II. Background and Challenges

In this section, we first introduce RPC and RDMA, then discuss the key points of implementing an easy-to-use and efficient RDMA-based RPC framework.

A. Datacenter RPC

RPC allows the machine to call functions on the remote machine just like calling local functions. Therefore, users only need to complete the logical implementation of the function body and do not need to pay attention to the underlying communication details. This simple and flexible programming semantics makes it widely used in modern distributed systems. However, the mainstream open source RPC frameworks in the industry are usually implemented based on the TCP/IP protocol stack. For example, gRPC and Apache Thrift do not have native RDMA support [8], which prevents them from fully utilizing the performance of hardware such as high-speed NIC in high-performance clusters.

A typical RPC framework mainly consists of a client, a server, and their respective stubs, which serialize and deserialize RPC requests/responses. The serialization phase is to convert the data object into a byte array, and the deserialization phase is to restore the byte array into a data object [9]. Of course, the serialization module will add some system overhead, especially when the data object itself is a byte array. Therefore, some research focuses on designing communication libraries based on application scenarios instead of directly using the RPC framework. However, the serialization module can parse custom structured data, has cross-platform features and good flexibility, so it is indispensable in common scenarios.

B. RDMA Programming Model

The use of RDMA is usually based on the InfiniBand protocol or the RoCEv2 protocol. Since the latter is compatible with Ethernet and requires lower hardware configuration, it has been widely used in modern data centers. RDMA can achieve hundreds of Gbps bandwidth and microsecond-level communication latency mainly due to its following two features: (1) Kernel bypass: Bypassing the OS kernel during data transmission avoids the overhead of data copying and context switching between user mode and kernel mode, and has very low latency. (2) CPU offloading: The local machine can directly access the memory of the remote machine and does not require the participation of the remote CPU, resulting in higher throughput.

RDMA builds a series of abstractions and interfaces for communication, which applications can use through user-mode libraries such as `ibverbs` and `rdmacm`. Specifically, communicators establish connections and communicate through a queue pair (QP). QP consists of a send queue (SQ) and a receive queue (RQ). RDMA allows different QPs to share a RQ and calls it a shared receive queue (SRQ). If inline mode is not considered, the local user-mode memory to be read and written by the RDMA NIC (RNIC) must be registered in advance. Each time the memory is registered, a pair of unique keys is generated, consisting of a local key (`lkey`) and a remote key (`rkey`). When accessing registered memory, `lkey` must be held for local access and `rkey` for remote access. Applications use various `verbs` operations by posting work requests (WR) into the QP. Among them, SEND/RECV are called two-sided operations. Before the sender posts a SEND WR to SQ, the receiver needs to post a RECV WR associated with a block of memory to RQ in advance. After RNIC receives the message, it writes it into this memory. WRITE/READ are called one-side operations. After the sender obtains the address and `rkey` of the remote memory, it can directly read and write the memory. Since each SQ and RQ needs to be bound to a completion queue (CQ), when the WR is executed by the RNIC, the RNIC will write a completion event (CQE) to the CQ. The application can know the execution status of the WR by obtaining the CQE.

RDMA provides three transmission modes: reliable connection (RC), unreliable connection (UC) and unreliable datagram (UD). Their main differences are as follows: (1) RC guarantees that data transmission is correct and reliable, but UC/UD does not. (2) RC/UC needs to establish a connection before communication and can only conduct one-to-one interaction. UD does not need to establish a connection, and one QP can interact with multiple QPs. (3) As shown in table I, these three modes have different support for `verbs` operations and the corresponding maximum transmission unit (MTU).

TABLE I
VERBS AND MTU IN DIFFERENT TRANSMISSION MODES

	SEND/RECV	WRITE	READ/ATOMIC	MTU
RC	✓	✓	✓	2GB
UC	✓	✓	✗	2GB
UD	✓	✗	✗	4KB

C. Design Challenges

How to better use the RDMA programming model in the RPC framework, we need to pay attention to the following questions: (1) How to choose the transmission mode and RDMA operation? (2) How to reduce the overhead caused by serialization? (3) How to obtain CQE? That is, sensing the completion of the send operation and the arrival of the remote message. (4) How to design a thread pool to handle heterogeneous function calculations?

III. Framework Design

In this section, we introduce the overall architecture and design details of LightRPC. Note that LightRPC uses RC mode because RC does not require upper-layer applications to achieve reliable transmission, reducing software complexity.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on September 15,2026 at 06:53:28 UTC from IEEE Xplore. Restrictions apply.

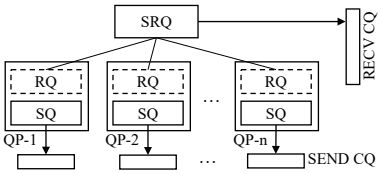


Fig. 1. RDMA queue model in SR.

```
   /* Build Server with SR */
1  SharedResource sr(local_ip, local_port, memory_size);
2  LightServer server(&ssr, num_worker_threads);
3  server.AddService(rpc_service);
4  server.BuildAndStart();

   /* Build Client with UR */
5  UniqueResource ur(local_ip, local_port, memory_size);
6  LightChannel chan(&ur, remote_ip, remote_port);

7  StubType stub(&chan);  // create stub
8  stub.method(request, reply);  // rpc call
```

Fig. 2. LightRPC user API and usage case.

A. Resource Abstraction

In terms of design, one of our main ideas is resource abstraction. We define the following two resource types:

- **UniqueResource** (UR): A unique resource for client use. It includes QP, SEND/RECV CQ, completion event channel, and pre-registered memory.
- **SharedResource** (SR): A shared resource for server use. Instead of a QP, it contains an SRQ and the rest of the resource types are the same as the UR.

Among them, SR allows different connections within the server process to share system resources. Taking the RDMA queue resource as an example, as shown in Figure 1, all QPs use the same RQ, that is, SRQ. Then, the SRQ is bound to the RECV CQ. Each SQ has its own SEND CQ.

Through resource abstraction, LightRPC has very convenient user interfaces. As shown in the code shown in Figure 2, when the user creates a server or client object, it must be bound to an already created resource object. In addition, users can set parameters such as the size of pre-registered memory and the number of worker threads.

B. Message Transmission

We implement two-side operations to build LightRPC. This is because one-side operations often require additional information interaction. For example, the sender needs to know the address and `rkey` of the receiver's memory, and bypassing the CPU is sometimes not what the receiver wants, etc. These problems may need to be solved through multiple round-trip times (RTTs), resulting in higher latency.

We call the byte array obtained by serializing a data object a message. We make an important observation about how to efficiently transmit a message: Pre-registered memory can improve the efficiency of message transmission. This is because, in non-inline mode, the memory read by RNIC must be registered. To meet this condition, we propose two methods: (1) After serializing the data object into a message, directly register the memory area where the message is located. (2) Register a block of memory in advance, obtain and use it when needed. Since memory registration is usually a time-consuming process [10], registering temporary memory frequently will lead to inefficiency, but pre-registered memory can avoid the overhead of memory registration at runtime.

Therefore, we use only one SEND operation for transmission and achieve low latency by using pre-registered memory. The process of sending a data object is: the sender first obtains the pre-registered memory, then directly serializes the data object into it, and finally posts the SEND WR to the RNIC. Note that if the serialization is implemented by the user rather than by the RPC framework, then the sending thread needs to copy the user message to the pre-registered memory. That is, the isolation of serialization and communication will additionally cause a temporary memory allocation (when the user performs serialization) and a user-mode memory copy (when the RPC framework sends a message).

C. Thread model

In LightRPC, we mainly set up two types of threads, and their functions and relationships are shown in the Figure 3. Below we introduce them in detail.

Polling thread, used to obtain CQE from RECV CQ, that is, to sense the arrival of remote messages. In fact, RDMA provides two ways to obtain CQE: (1) Busy polling: The polling thread quickly obtains CQE in CQ by continuously calling `ibv_poll_cq`. This method has very low latency, but when no CQE is generated, it will cause a waste of CPU. (2) Event notification: After CQ is bound to a completion event channel, the generation of CQE will generate an event in the channel. The polling thread will obtain events through the blocking interface `ibv_get_cq_event`. When no event occurs, the polling thread enters the blocking state until it is awakened when a CQE is generated. Therefore, this method saves CPU resources, but the latency is relatively high. We combine the advantages of both and design an adaptive polling mode. First, at the client, due to the low latency of the RDMA network, the client directly senses the arrival of the response through busy polling after sending the RPC request. Secondly, on the server side, since we do not know when client requests will arrive, if we always use busy polling, it will cause a waste of CPU. On the contrary, if we always use the event notification mode, in the case of intensive requests, frequent interruptions can lead to inefficiency. Therefore, we first let the polling thread work in event notification mode. Once a request arrives, it switches to busy polling mode. If the busy polling does not get the request for a certain period of time, it switches to event notification mode again to save CPU resource. Note that for CQE in SEND CQ, the sending thread calls `ibv_poll_cq` to obtain it after sending the request.

Worker thread pool, used for server-side RPC request processing. In terms of design, we allow each worker thread to have an independent task queue to avoid excessive lock competition caused by all worker threads sharing the task

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on September 15,2026 at 06:53:28 UTC from IEEE Xplore. Restrictions apply.

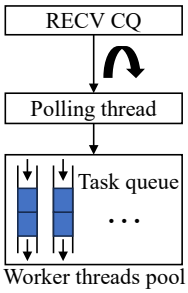


Fig. 3. The server's thread model.

queue. In addition, if the round-robin is used to divide tasks into various worker threads, then in scenarios where the processing times of different tasks vary greatly, the problem of head-of-line blocking may occur. To avoid this problem, we calculate the number of tasks remaining in the task queue by recording the number of tasks assigned and the number of completed tasks for each worker thread. During the scheduling process, we assign tasks to the worker thread with the smallest number of remaining tasks to achieve dynamic load balancing.

D. Memory Management

Since memory registration is very time-consuming, if the memory needs to be temporarily registered to send any message, the communication latency will be greatly increased, so we provide pre-registered memory for the sending and receiving of messages. Specifically, the client will pre-register two memory blocks of size *T*, one for sending messages and the other for receiving messages. The server will register a memory pool and divide it into many blocks of *T* bytes. We use a lock-free queue to store the address of each Block to achieve high-concurrency application and return.

In terms of memory resource recycling, we use two methods to handle it: (1) For the block that stores the received message, the receiving thread actively returns it after acquiring the message. (2) For the memory that stores sent messages, since `ibv_post_send` is asynchronous, we need to sense that the block has been used by polling the CQE in SEND CQ. In fact, the address of the block where we store the message is recorded in the `wr_id` field of `ibv_send_wr`, so that when we obtain the CQE, we can easily obtain the memory address through the `wr_id` field of `ibv_wc` and return it.

IV. Evaluation

A. Experimental Setup

Evaluation Platform: We use two nodes for multi-connection testing. Each node is configured with two 18-core Intel Xeon Gold 6150 CPUs, 192GB of memory, and a 100Gbps Mellanox ConnectX-4 NIC. They are installed with CentOS 7 and Linux kernel version 3.10.0. On the experimental setup, we start different numbers of user threads on the client node, and each thread establishes a connection with the server node and performs uninterrupted RPC calls for a duration time. Note that we use echo testing, that is, the

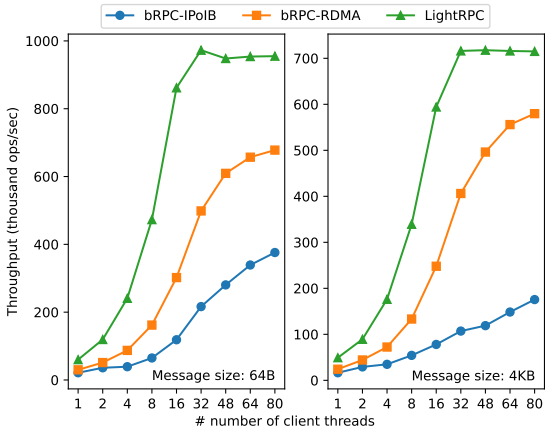


Fig. 4. Throughput for LightRPC and bRPC

RPC response has exactly the same data as the request. We test how the system throughput and latency changes with the number of threads under RPC requests of different sizes.

Comparison: We compare LightRPC with bRPC [11] because bRPC is currently the industrial-level open source RPC framework with the best comprehensive performance, and more importantly, it supports the RDMA protocol. During the experiment, we use IPoIB (IP protocol based on IB network card) and RDMA mode of bRPC version 1.5.0.

B. Experimental Results

Throughput: As shown in the figure4, as the number of connections increases, the throughput gap between LightRPC and bRPC gradually increases. This is mainly due to the fact that LightRPC not only has better performance for single connection, but also has better scalability. It is worth noting that LightRPC reaches maximum throughput at 32 connections, and bRPC stabilizes at 80 connections. Finally, when the message size is 64B, the throughput of LightRPC is 2.5x and 1.4x that of bRPC-IPoIB and bRPC-RDMA respectively. When the message size is 4KB, it is 4x and 1.2x respectively.

Latency: Figure 5 and Figure 6 show the changes in median latency and P99 latency respectively. It can be seen that as the number of connections increases, the median latency and P99 latency of bRPC continue to increase, and there is no stable trend (except when the message size is 4KB, bRPC-IPoIB finally reaches stability), but the latency of LightRPC increases relatively slow and reaches stability after 32 threads. This is mainly because in high concurrency situations, LightRPC's polling thread will automatically work in busy polling mode, and message sensing is very fast, while bRPC always uses event notification mode. Finally, taking the 64B message as an example, the tail latency of LightRPC is only 13% of bRPC-IPoIB and 28% of bRPC-RDMA.

V. Related Work

Whether in academia or industry, there is a series of work to build an RPC framework based on RDMA.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on September 15,2026 at 06:53:28 UTC from IEEE Xplore. Restrictions apply.

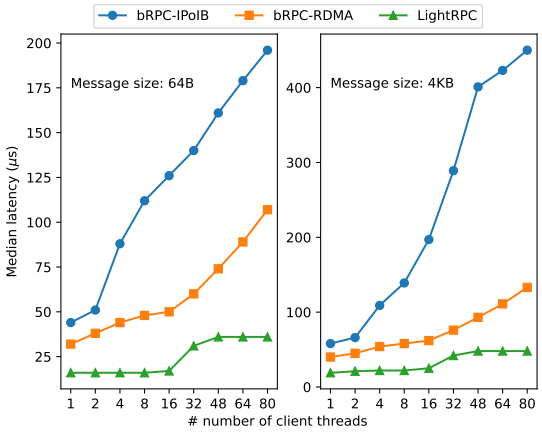


Fig. 5. Median latency for LightRPC and bRPC

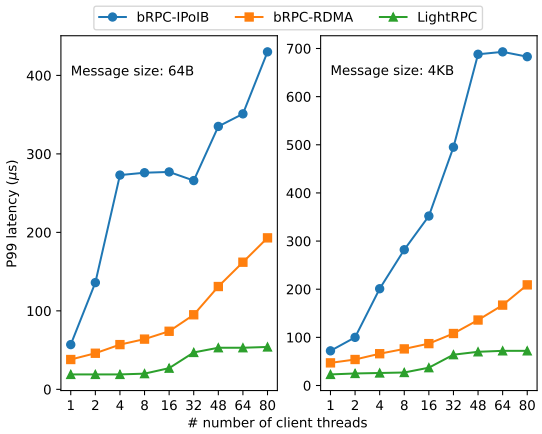


Fig. 6. P99 latency for LightRPC and bRPC

UD-based work: HERD [12] uses the WRITE operation of UC mode at the client to send the request, and the SEND operation of UD mode at the server to reply the response. FaSST [13] uses the SEND operation of UD mode for both request and response sending. This is mainly due to the one-to-many feature of the UD mode, which makes it more scalable. eRPC [4] is implemented based on the UDP protocol and the UD mode of RDMA, and provides reliable transmission by implementing functions such as congestion control and packet loss retransmission at the software layer. The goal of eRPC is to provide low latency for small messages, high bandwidth for large messages, and achieve scalability in large-scale clusters.

RC-based work: FaRM [14] uses the WRITE operation to write the message into the receiver's ring buffer. By letting multiple threads share the QP and using 2GB huge page memory, FaRM reduces the amount of data to be cached in the RNIC. RFP [6] defines a remote fetching paradigm that allows clients to use the WRITE operation to send requests and READ operation to obtain responses. Since the client does not know when the response is generated, RFP first allows the client to actively read, and when the number of failures reaches a certain threshold, it switches to the server's active reply mode. ScaleRPC [7] finds that the reasons why outbound and inbound operations are not scalable are: the RNIC cache size is limited and the CPU last-level cache size is limited. ScaleRPC balances RNIC cache contention through connection grouping and reduces memory consumption and CPU last-level cache misses through virtual mapping.

REFERENCES

[1] Bohong Zhu, Youmin Chen, Qing Wang, Youyou Lu, and Jiwu Shu. Octopus+: An rdma-enabled distributed persistent memory file system. *ACM Transactions on Storage (TOS)*, 17(3):1–25, 2021.

[2] Yanzhe Chen, Xingda Wei, Jiaxin Shi, Rong Chen, and Haibo Chen. Fast and general distributed transactions using rdma and htm. In *Proceedings of the Eleventh European Conference on Computer Systems*, pages 1–17, 2016.

[3] Jilong Xue, Youshan Miao, Cheng Chen, Ming Wu, Lintao Zhang, and Lidong Zhou. Fast distributed deep learning over rdma. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–14, 2019.

[4] Anuj Kalia, Michael Kaminsky, and David Andersen. Datacenter rpcs can be general and fast. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 1–16, 2019.

[5] Sumit Kumar Monga, Sanidhya Kashyap, and Changwoo Min. Birds of a feather flock together: Scaling rdma rpcs with flock. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 212–227, 2021.

[6] Maomeng Su, Mingxing Zhang, Kang Chen, Zhenyu Guo, and Yongwei Wu. Rfp: When rpc is faster than server-bypass with rdma. In *Proceedings of the Twelfth European Conference on Computer Systems*, pages 1–15, 2017.

[7] Youmin Chen, Youyou Lu, and Jiwu Shu. Scalable rdma rpc on reliable connection with efficient resource sharing. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–14, 2019.

[8] Rajarshi Biswas, Xiaoyi Lu, and Dhabaleswar K Panda. Accelerating tensorflow with adaptive rdma-based grpc. In *2018 IEEE 25th International Conference on High Performance Computing (HiPC)*, pages 2–11. IEEE, 2018.

[9] Mingyu Wu, Shuaiwei Wang, Haibo Chen, and Binyu Zang. Zero-change object transmission for distributed big data analytics. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 137–150, 2022.

[10] Philip Werner Frey and Gustavo Alonso. Minimizing the hidden cost of rdma. In *2009 29th IEEE International Conference on Distributed Computing Systems*, pages 553–560. IEEE, 2009.

[11] Zhaogeng Li, Ning Liu, and Jiaoren Wu. Toward a production-ready general-purpose rdma-enabled rpc. In *Proceedings of the ACM SIGCOMM 2019 Conference Posters and Demos*, pages 27–29, 2019.

[12] Anuj Kalia, Michael Kaminsky, and David G Andersen. Using rdma efficiently for key-value services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, pages 295–306, 2014.

[13] Anuj Kalia, Michael Kaminsky, and David G Andersen. Fasst: Fast, scalable and simple distributed transactions with two-sided rdma datagram rpcs. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 185–201, 2016.

[14] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. Farm: Fast remote memory. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 401–414, 2014.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on September 15,2026 at 06:53:28 UTC from IEEE Xplore. Restrictions apply.